- **College Name : Pune Vidyarthi Griha's COLLEGE OF ENGINEERING, TECHNOLOGY AND MANAGEMENT,**
- **Department Name: Department of Artificial Intelligence and Data Science**
- **Student Name & Roll No.**
- **Class & Batch: BE**

EXPERIMENT 6: Data Aggregation

Problem Statement: Analyzing Sales Performance by Region in a Retail Company

Prerequisites

pip install pandas numpy matplotlib seaborn

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages
(3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.13/dist-packages (0.13.2)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.13/distpackages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2026.4)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages
(from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (4.66.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages
(from matplotlib) (26.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from
matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/distpackages
(from matplotlib) (3.3.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from
python-dateutil>=2.8.2->pandas) (1.17.0)
```

Step 0: Generate Sample Retail_Sales_Data.csv

```
import pandas as pd
import numpy as np
import os
```

```
if not os.path.exists("Retail_Sales_Data.csv"):
    np.random.seed(42)
n = 300

    regions = ['West', 'South', 'North', 'East', 'Central']
    categories = ['Electronics', 'Apparel', 'Home & Kitchen', 'Groceries', 'Beauty & Personal Care']
payment_modes = ['UPI', 'Credit Card', 'Debit Card', 'Net Banking', 'Cash']

    # Assign regional weights so top-performing regions emerge clearly     region_choices
= np.random.choice(regions, n, p=[0.30, 0.25, 0.20, 0.15, 0.10])     category_choices =
np.random.choice(categories,  n,  p=[0.25,  0.25,  0.20,  0.20,  0.10])         quantities  =
np.random.randint(1, 8, n)

    # Set realistic unit price ranges per category (in INR)
base_prices = {
        'Electronics': (8000, 45000),
        'Apparel': (800, 4500),
        'Home & Kitchen': (1200, 12000),
        'Groceries': (250, 2500),
        'Beauty & Personal Care': (300, 3000)
    }

    unit_prices = [
        np.round(np.random.uniform(*base_prices[cat]), 2)
for cat in category_choices
    ]

    sales_amounts = np.round(quantities * np.array(unit_prices), 2)

    df_sample = pd.DataFrame({
        'Transaction_ID': [f'TXN-{10000+i}' for i in range(n)],
        'Transaction_Date': pd.date_range(start='2026-01-01', periods=n, freq='18h').strftime('%Y-%m-
%d'),
        'Region': region_choices,
        'Product_Category': category_choices,
        'Quantity_Sold': quantities,
        'Unit_Price': unit_prices,
        'Sales_Amount': sales_amounts,
        'Payment_Mode': np.random.choice(payment_modes, n, p=[0.45, 0.25, 0.15, 0.10, 0.05])
    })

    # Inject a few missing values to demonstrate safe pre-aggregation checks
df_sample.loc[10:12, 'Sales_Amount'] = np.nan

    df_sample.to_csv("Retail_Sales_Data.csv",                 index=False)
print("Sample  'Retail_Sales_Data.csv'  generated  successfully.\n")
Sample 'Retail_Sales_Data.csv' generated successfully.
```

```
import pandas as pd import
numpy as np import
matplotlib.pyplot as plt import
seaborn as sns




# Set visual theme sns.set_theme(style="whitegrid")
plt.rcParams['font.size'] = 10
plt.rcParams['axes.titlesize'] = 12
plt.rcParams['axes.titleweight'] = 'bold'
```

TASK 1 & 2: Import and Explore the Dataset

```
df = pd.read_csv("Retail_Sales_Data.csv")

print("--- Task 1 & 2: Dataset Structure & Exploration ---") print(f"Dataset
Shape: {df.shape}")
print("\nColumn Data Types & Non-Null Counts:")
print(df.info()) print("\nSummary Statistics:")
print(df.describe().round(2)) print("\nFirst 5
Records:") print(df.head())
--- Task 1 & 2: Dataset Structure & Exploration ---
Dataset Shape: (300, 8)

Column Data Types & Non-Null Counts:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300 entries, 0 to 299 Data
columns (total 8 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
0   Transaction_ID    300 non-null    object
1   Transaction_Date  300 non-null    object
2   Region            300 non-null    object
3   Product_Category  300 non-null    object
4   Quantity_Sold     300 non-null    int64
5   Unit_Price        300 non-null    float64
6   Sales_Amount      297 non-null    float64  7   Payment_Mode      300 non-null    object
    dtypes: float64(2), int64(1), object(5)
memory usage: 18.9+ KB
None

Summary Statistics:
       Quantity_Sold  Unit_Price  Sales_Amount
count         300.00      300.00        297.00
mean            4.09     9348.21      38362.51
std             2.06    12439.86      61097.36
min             1.00      279.25        349.65
25%             2.00     1647.57       5665.26
```

```
50%             4.00     3001.03      11569.80
75%             6.00    10956.04      42937.86
max             7.00    44923.56     307504.12

First 5 Records:
  Transaction_ID Transaction_Date   Region Product_Category  Quantity_Sold  \
0      TXN-10000       2026-01-01    South      Electronics              5
1      TXN-10001       2026-01-01  Central   Home & Kitchen              2
2      TXN-10002       2026-01-02    North   Home & Kitchen              4
3      TXN-10003       2026-01-03    North   Home & Kitchen              2    4      TXN-
       10004        2026-01-04     West         Groceries               5       Unit_Price
       Sales_Amount Payment_Mode
0     40440.33     202201.65          UPI
1     4894.48       9788.96          UPI
2     10069.58      40278.32          UPI
3     2394.82       4789.64          UPI
4     2154.52      10772.60          UPI
```

TASK 3: Identify & Clean Relevant Variables

```
relevant_cols = ['Region', 'Product_Category', 'Quantity_Sold', 'Sales_Amount'] print("\n---
Task 3: Checking Relevant Variables ---")
print("Missing values in key columns:\n", df[relevant_cols].isnull().sum())

# Recompute or impute any missing Sales_Amount values before aggregation if
df['Sales_Amount'].isnull().sum() > 0:    if 'Unit_Price' in df.columns:
df['Sales_Amount'] = df['Sales_Amount'].fillna(df['Quantity_Sold'] * df['Unit_Price'])
else:
    df['Sales_Amount'] = df['Sales_Amount'].fillna(df['Sales_Amount'].median())

# Standardize string formatting in grouping columns df['Region']
= df['Region'].astype(str).str.strip().str.title()
df['Product_Category'] = df['Product_Category'].astype(str).str.strip().str.title()
```

```
--- Task 3: Checking Relevant Variables --- Missing
values in key columns:
 Region               0 Product_Category
0
Quantity_Sold        0
Sales_Amount         3 dtype:
int64
```

TASK 4: Group by Region & Calculate Total Sales

```
regional_sales = df.groupby('Region', as_index=False).agg(
  Total_Transactions=('Transaction_ID', 'count'),
  Total_Units_Sold=('Quantity_Sold', 'sum'),
  Total_Sales_Amount=('Sales_Amount', 'sum'),
  Average_Order_Value=('Sales_Amount', 'mean') ).round(2)

# Sort descending by Total_Sales_Amount
```

```
regional_sales = regional_sales.sort_values(by='Total_Sales_Amount',
ascending=False).reset_index(drop=True)

# Add percentage share of overall revenue
total_company_sales = regional_sales['Total_Sales_Amount'].sum()
regional_sales['Revenue_Share_Pct'] = ((regional_sales['Total_Sales_Amount'] / total_company_sales) *
100).round(2)

print("\n--- Task 4: Sales Aggregated by Region ---") print(regional_sales.to_string(index=False))
```

```
--- Task 4: Sales Aggregated by Region ---
 Region   Total_Transactions   Total_Units_Sold   Total_Sales_Amount   Average_Order_Value
Revenue_Share_Pct
   West                  95               406          3933559.22              41405.89
33.91
  South                  70               277          2469182.68              35274.04
21.29
  North                  59               229          1892976.70              32084.35
16.32
   East                  47               202          1733804.53              36889.46
14.95
Central                  29               113           1569625.60               54125.02
13.53
```

TASK 6: Identify Top-Performing Regions

```
top_region = regional_sales.iloc[0] top_3_regions
= regional_sales.head(3)

print("\n--- Task 6: Top-Performing Regions ---")
print(f"Top-Performing Region : {top_region['Region']} (₹{top_region['Total_Sales_Amount']:,.2f} |
{top_region['Revenue_Share_Pct']}% of total revenue)") print("\nTop
3 Regions by Sales Amount:")
print(top_3_regions[['Region', 'Total_Sales_Amount', 'Revenue_Share_Pct']].to_string(index=False))
```

```
--- Task 6: Top-Performing Regions ---
Top-Performing Region : West (₹3,933,559.22 | 33.91% of total revenue)

Top 3 Regions by Sales Amount:
Region  Total_Sales_Amount  Revenue_Share_Pct
  West          3933559.22              33.91
 South          2469182.68              21.29
 North          1892976.70              16.32
```

TASK 5: Visualize Sales Distribution by Region (Bar & Pie Chart)

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5.5))
fig.suptitle('Task 5: Regional Sales Performance Distribution', fontsize=14, fontweight='bold')

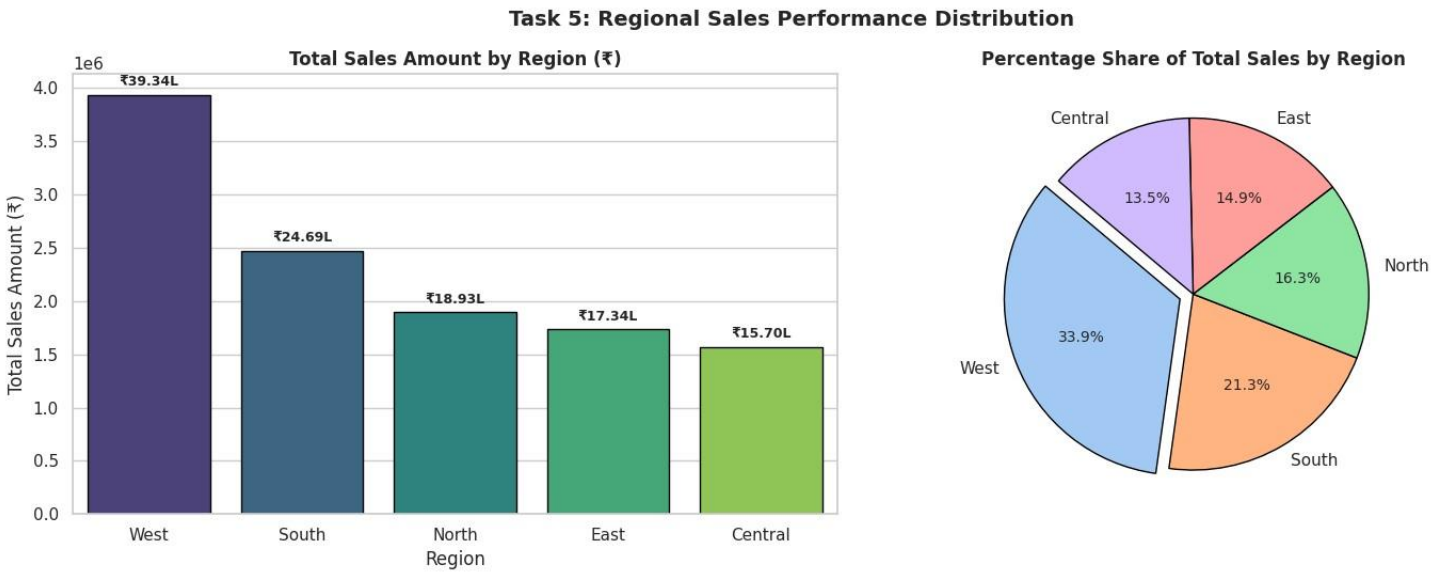
# 5A. Bar Plot of Total Sales by Region
bars = sns.barplot(
data=regional_sales,
  x='Region',
  y='Total_Sales_Amount',
  hue='Region',
palette='viridis',
edgecolor='black',
ax=ax1,   legend=False
)
ax1.set_title('Total Sales Amount by Region (₹)') ax1.set_xlabel('Region')
ax1.set_ylabel('Total Sales Amount (₹)')



# Add value labels on top of each bar for
bar in ax1.patches:
  height = bar.get_height()
ax1.annotate(
f'₹{height/1e5:.2f}L',
     xy=(bar.get_x() + bar.get_width() / 2, height),
xytext=(0, 4),
     textcoords="offset points",
     ha='center', va='bottom', fontsize=9, fontweight='bold'
  )

# 5B. Pie Chart of Regional Revenue Share
explode_vals = [0.08 if i == 0 else 0 for i in range(len(regional_sales))]  # Highlight #1 region
ax2.pie(   regional_sales['Total_Sales_Amount'],   labels=regional_sales['Region'],
  autopct='%1.1f%%',
startangle=140,
explode=explode_vals,
  colors=sns.color_palette('pastel'),
  wedgeprops={'edgecolor': 'black', 'linewidth': 1}
)
ax2.set_title('Percentage Share of Total Sales by Region')

plt.tight_layout() plt.show()
```



TASK 7: Group by Region and Product Category

```
region_category_sales = df.groupby(['Region', 'Product_Category'], as_index=False).agg(
   Total_Units_Sold=('Quantity_Sold', 'sum'),
   Total_Sales_Amount=('Sales_Amount', 'sum') ).round(2)

print("\n--- Task 7A: Sales Aggregated by Region & Product Category (Long Format) ---")
print(region_category_sales.head(10).to_string(index=False))



# Create a Pivot Table (Cross-Tabulation) for easy multi-dimensional comparison & stacked plotting
sales_pivot = df.pivot_table(    index='Region',    columns='Product_Category',
values='Sales_Amount',    aggfunc='sum',
   fill_value=0
).round(2)

# Reorder rows to match the top-performing region order
sales_pivot = sales_pivot.loc[regional_sales['Region']]

print("\n--- Task 7B: Region vs. Product Category Pivot Table (₹) ---")
print(sales_pivot)
```

```
--- Task 7A: Sales Aggregated by Region & Product Category (Long Format) ---
 Region       Product_Category  Total_Units_Sold  Total_Sales_Amount
Central                Apparel                23            75270.28
Central Beauty & Personal Care                11            11570.65
Central            Electronics                48          1285512.26
Central              Groceries                 3             7152.06
Central         Home & Kitchen                28           190120.35
   East                Apparel                53           124750.32
   East Beauty & Personal Care                 9            13479.79
   East            Electronics                47          1372327.27
```

```
   East                Groceries                   56             53568.80
East         Home & Kitchen                 37           169678.35

--- Task 7B: Region vs. Product Category Pivot Table (₹) ---
Product_Category    Apparel  Beauty & Personal Care  Electronics  Groceries  \
Region
West              185419.09               103905.72   3040866.68  120638.42
South             160066.92                64396.48   1853250.30   80358.53
North             160539.70                71742.37   1279002.57   47223.10
East              124750.32                13479.79   1372327.27   53568.80    Central
75270.28               11570.65   1285512.26    7152.06

Product_Category  Home & Kitchen
Region
West                   482729.31
South                  311110.45
North                  334468.96
East                   169678.35
Central                190120.35
```

TASK 8: Grouped Bar Plot & Stacked Bar Plot (Region vs. Category)

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Task 8: Sales Comparison Across Regions and Product Categories', fontsize=14, fontweight='bold')




# 8A. Grouped Bar Plot using Seaborn
sns.barplot(
    data=region_category_sales,
x='Region',    y='Total_Sales_Amount',
hue='Product_Category',
order=regional_sales['Region'],
    palette='Set2',
edgecolor='black',    ax=ax1
)
ax1.set_title('Task 8A: Grouped Bar Plot (Region × Product Category)') ax1.set_xlabel('Region')
ax1.set_ylabel('Total Sales Amount (₹)')
ax1.legend(title='Product Category', loc='upper right')

# 8B. Stacked Bar Plot using Pandas/Matplotlib on the Pivot Table
sales_pivot.plot(    kind='bar',    stacked=True,
colormap='tab20c',    edgecolor='black',    width=0.65,    ax=ax2
)
ax2.set_title('Task 8B: Stacked Bar Plot of Category Composition per Region') ax2.set_xlabel('Region')
```

```
ax2.set_ylabel('Cumulative Sales Amount (₹)') ax2.tick_params(axis='x',
rotation=0)
ax2.legend(title='Product Category', loc='upper right')

plt.tight_layout() plt.show()
```

